ChatGPT

The Hidden Cost of Ephemeral Testing and the Case for Automation

by Dinis Cruz and ChatGPT Deep Research

Executive Summary

Software developers **always test their code** – but **not always in a repeatable way**. Often, the only tests run are the quick, informal checks a developer performs in their head or manually in a browser or console. These **"ephemeral tests"** are executed once and then forgotten, leaving no trace. Such practice comes at a **steep hidden cost**. Every time a developer manually verifies a change (for example, by running the application and clicking through a scenario), they incur a **context-switching penalty** and lose an opportunity to build a permanent safety net. Over days and months, these costs accumulate: manual re-testing of old features, fear of breaking existing behavior, and slower debugging of regressions. In contrast, **automated tests** turn those one-off checks into lasting assets that run quickly and consistently. Automated test suites catch side effects of changes early and guard the user experience from future regressions, enabling developers to maintain speed with confidence.

The key insight is that **lack of visible tests doesn't mean testing isn't happening** – it means testing is happening inefficiently. Developers practicing Ephemeral Test-Driven Development (ETDD) "write" tests in their mind and execute them with their hands, like scribbling on a whiteboard and erasing it after each use. This ad-hoc approach *feels* fast in the moment, but it is **deceptively expensive over time**. Each manual test is wasted effort that must be repeated (or risks being skipped later), whereas an automated test, once written, can run endlessly at virtually no extra cost [1]. Teams that embrace a culture of capturing these checks as code see **compound benefits**: every new test makes the next code change safer and faster, creating a positive feedback loop of confidence and velocity. On the other hand, teams averse to testing often operate in environments where testing is treated as a tick-box metric rather than a quality practice. In such cases, developers write minimal or meaningless tests just to satisfy coverage requirements, defeating the purpose [2] [3]. This paper argues that the focus must shift from **"writing tests for metrics"** to **"automating tests for insight"** – i.e. capturing the most efficient way to execute code and verify its behavior, so that any change's impact is immediately known. Achieving this requires investment in testing infrastructure and developer experience: if writing tests is cumbersome or slow, that friction needs to be addressed (through better design, tools, or training) rather than bypassed. Modern tools like Wallaby.js and NCrunch demonstrate that with the right environment, developers can code and test almost in lockstep, dramatically reducing context switches and error rates. Ultimately, a healthy testing culture, combined with strategic awareness (e.g. using Wardley Maps to adjust practices to the maturity of the project), allows teams to deliver business value faster **without** sacrificing quality. In summary, **automated testing is not a tax on development speed – it is a powerful enabler of sustainable velocity**, and its absence incurs a far greater cost in the long run.

Ephemeral Test-Driven Development (ETDD): The Hidden Cost of "Testing in Your Head"

All developers perform testing during coding – the question is **how** they do it. In many teams, the predominant method is what we can call *Ephemeral Test-Driven Development (ETDD)*: developers mentally formulate test cases or manually try out their code changes, observe the result once, then move on. Perhaps they run the program and click a button to see if a feature works, or add a temporary `console.log` to inspect output. These quick experiments are essentially **one-off tests** that vanish immediately after execution (like writing on a blackboard that gets erased). The functionality might be "tested" in that moment, but because the test isn't recorded anywhere, **the knowledge gained is not retained**. Next time that code is modified – whether in an hour or a month – the same manual steps will have to be repeated to verify it (if they aren't forgotten altogether). In the words of one engineer, *"If I write code and test it manually, I regard this as wasted effort… the code worked when I did the tests, but if I change the code I'm just going to have to repeat those tests"* [4]. In contrast, by automating the test, *"I have a suite of tests that I can re-run any time to verify the code still works… years to come"* [5]. In short, **manual tests have "zero residual value," whereas automated tests provide lasting regression coverage** [6].

Beyond the obvious repetition, ETDD carries significant **context-switching costs**. A developer verifying code manually must stop writing code, shift to a runtime environment, set up test conditions by hand, and then mentally diff the outcome against expectations. Studies show that frequent task-switching exacts a toll on productivity; for example, extended bug fix cycles increase development costs partly due to constant context switching and rework [7]. Even a "quick" manual test that takes, say, 30 seconds disrupts the coder's flow. If their edit-compile-run cycle isn't tight, they'll naturally try to amortize the cost by batching more code changes between tests – which is *even riskier*. Developers without an automated test harness often end up making larger edits (waiting maybe 5-10 minutes or more before manually checking anything) to avoid slow feedback loops. This creates a dangerous snowball effect: **the longer a developer goes between tests, the more intertwined changes become, and the harder it is to isolate or fix problems**. It's no surprise that in projects with slow or no tests, debugging sessions and regression bugs multiply, and adding tests later becomes daunting. One author observes that skipping tests may seem to save time for a while, but as features grow, *"the burden of (manual) regression testing grows exponentially… soon you compromise between quality, cost, and time"* [8]. Initially it might be *"two or three times longer to write a good unit test than to test your feature by hand,"* but after just a few cycles *"you will be ahead – and it will only get better"* [9]. In other words, **the cost of each ephemeral test compounds over time**, whereas the cost of an automated test diminishes with each reuse.

Perhaps the most insidious cost of ETDD is **the impact on team knowledge and future development**. Because ephemeral tests leave no artifact, new team members or even your future self have no way of knowing how a piece of code was verified. The system accumulates "tribal knowledge" of what manual steps need to be run after certain changes – knowledge that often lives only in individual heads or gets lost. Michael Feathers famously defined *legacy code* as "code without tests" [10]. Code without an automated test suite tends to induce fear and caution: developers are **"afraid of touching it and breaking stuff"**, making them hesitate to refactor or add new features [10]. In contrast, a robust test suite provides a safety net that encourages cleanups and improvements because any breaking change will be caught early. Thus, a codebase heavily reliant on undocumented manual testing **rapidly turns into "legacy code" – brittle, mysterious, and resistant to change** [10]. In summary, ETDD's hidden costs include wasted effort, frequent context switches, larger batch sizes (leading to harder debugging), and a gradual build-up of fear and technical debt. Recognizing this unseen drag on productivity is the first step towards embracing a better approach.

Everything Needs a "Test": Capturing Change and Side Effects

A common misconception is that testing is only about unit tests or only relevant to certain types of code. In reality, **any change to any part of a software system can benefit from a test** – whether it's low-level code, a server configuration, a database migration, a continuous integration (CI) pipeline script, or a business logic tweak. The purpose of testing is to **capture "what changed" and ensure we understand (and control) the side effects of that change**. Every piece of the system – from backend modules to front-end UI, from infrastructure as code to deployment processes – can have unintended ripple effects when modified. Having a spectrum of automated tests (unit, integration, end-to-end, performance, security, etc.) is about creating a **safety net that flags unexpected consequences** before they reach the end user. When a team has this safety net, they can innovate and refactor quickly, confident that if something important breaks, a test will alert them. This is ultimately what gives development speed *without* sacrificing quality: you can move fast **and** not break things.

Crucially, the goal of testing isn't just to assert that *a component works in isolation* – it's to ensure that *the system's behavior as a whole remains correct as it evolves*. High test coverage (especially across integration points) means developers spend less time firefighting regressions and more time building new value. Empirical evidence backs this up: organizations that integrate automated testing early and extensively see dramatically fewer production defects (one study found "early testing" practices reduced post-release defects by 75% compared to traditional late testing [11]). By catching issues at the source, you avoid the exponentially higher costs of fixing bugs in later stages or after release [7]. Furthermore, comprehensive tests guard the **user experience** – they ensure that new changes don't break existing features or degrade performance, preserving the quality that users expect. In a sense, automated tests act as proactive user advocates, continuously checking that the product still behaves as intended after every change.

It's important to emphasize that **testing is a means to an end (quality and confidence), not an end in itself**. We write tests to gain insight into our software's behavior. This is why teams that treat testing as a mere paperwork exercise ("write some tests because we have to") often miss the real benefits. When done right, testing becomes intertwined with design and development. It can even drive better design: code written with testing in mind tends to be more modular and clear, whereas code never tested often ends up tightly coupled and opaque [12]. As Feathers noted, code without tests usually has **high coupling and unclear cohesion, making it messy and risky to change** [12]. By contrast, thinking about how to test a component forces developers to clarify its responsibilities and boundaries – leading to cleaner architecture. In summary, **every meaningful element of a software system should have some form of automated check**, proportionate to its impact. This doesn't mean writing trivial tests for the sake of it, but rather ensuring that for each change (be it a code commit, a config change, or a pipeline update) you have a reliable way to verify it does what it should *and nothing else*. Teams that internalize this habit essentially deploy a "continuous guardrail" around their work – one that speeds them up by eliminating the need for lengthy manual regressions or anxious guessing about what might break. As one expert put it, **"you don't have time *not* to do testing"**, because the short-term saved minutes will turn into long-term lost hours or days when defects slip through [13] [9].

The "Metric Trap": Why Focusing on Coverage Percentages Misses the Point

Teams that are **"allergic" to writing tests** are often victims of the wrong incentives. In some organizations, upper management decrees testing goals in terms of numbers – e.g. "all code must have 80% coverage." Testing becomes a metric to hit, divorced from the actual value it's supposed to provide. This approach is counterproductive and breeds cynicism among developers. As one DevOps veteran

warns, **“Measuring humans changes behavior – often not how we’d like. The most dangerous metric I’ve found is code coverage”** [14]. Mandating a coverage number tends to produce exactly what you’d expect: tests that satisfy the letter of the requirement but not the spirit. Developers under pressure to reach, say, 80% coverage might write a flurry of superficial tests that execute code paths without truly asserting correct behavior. It has been *“repeatedly demonstrated that imposing [coverage] as a measure of quality results in dangerously poor ‘tests’.”* [2] In one documented case, a company tied bonuses to hitting an 80% coverage target. Sure enough, the teams “achieved” the goal – on paper. But an audit revealed that **over 25% of the tests had no assertions at all** – they tested nothing! [3] Developers had written empty tests simply to make the coverage tool happy, gaming the metric while providing zero protection against bugs. *“They had paid people… to write tests that tested nothing at all,”* recounts Dave Farley of this fiasco [3]. Such meaningless tests create a false sense of security and can be worse than having no tests, because they may lull the team into thinking quality is higher than it really is. As Bryan Finster quips, *“What value is 80% coverage you do not trust? It’s lower value than 0% coverage – at least with 0% you know there are no tests.”* [15]

The **core problem with the metric-focused mindset** is that it inverts the relationship between testing and quality. Code coverage should be viewed as a *side effect* of thorough testing, not a goal by itself [2]. High coverage can arise naturally when a team writes a lot of meaningful tests, but simply chasing coverage doesn’t guarantee those tests have any depth. In fact, teams that are *forced* to care about the number often end up optimizing for the wrong thing – they’ll test trivial getters and setters, or add tests that execute code without verifying outcomes, just to bump the percentage. Meanwhile, more complex scenarios that truly need testing (because they might fail in subtle ways) could be ignored if they’re hard to automate, since the metric doesn’t differentiate *which* 80% of code is covered. This is the **“coverage fallacy”**: equating quantity of test code with quality of testing. Goodhart’s law (“when a measure becomes a target, it ceases to be a good measure”) applies here. As soon as developers feel they’re being judged by the percent number rather than by fewer bugs or faster iteration, their incentive is to make the number look good – even at the expense of real defect prevention.

None of this is to say metrics are useless; they can be a useful diagnostic tool (for example, coverage can identify untested areas to consider). But they must not override engineering common sense. A much healthier approach, as experts suggest, is to encourage practices that lead to high coverage as a *byproduct* – e.g. test-driven development or always writing a test when fixing a bug – rather than making the number itself the objective [16] [17]. Teams with a strong quality culture often achieve 85-95% coverage without explicitly measuring it, simply because they test thoroughly by habit [16]. Those teams *“aren’t chasing the metric… it’s only a side effect of their quality process,”* and they don’t need a mandate [16]. On the other hand, teams lacking training or ownership in testing will not magically become quality-focused because of a coverage edict – as we saw, they’ll either ignore it or game it [18]. In organizations where management fixates on coverage, developers usually sense a disconnect: tests feel like busywork for pleasing a dashboard, not a helpful part of development. To fix test allergy, **leaders must shift emphasis away from raw numbers and toward tangible outcomes**: fewer production bugs, faster cycle times, and higher confidence in deployments. In practical terms, this means giving teams the freedom and support to write meaningful tests (even if it means coverage is 70% instead of 90%, as long as critical paths are tested well), and evaluating success by how reliably and quickly the team can deliver changes. When developers see that tests are truly there to make *their* lives easier (catching their mistakes, preventing late-night fire drills) rather than to satisfy some arbitrary KPI, they are far more likely to embrace testing. A poignant summary from Finster: *“We don’t need to encourage testing [via mandates]. We need to encourage the continuous and sustainable delivery of valuable solutions. We need metrics that create the right balance… High coverage of low-quality tests is not the goal.”* [18] [19] In short, **stop managing to test metrics, and start investing in test *capability***.

Investing in Test Infrastructure and Developer Experience

If writing tests is consistently seen as pain, that's a smell – not to ignore, but to investigate. Many teams that avoid testing do so because **their testing process is genuinely tedious or slow**. This is often a tooling or infrastructure problem, not an excuse to give up on testing. The solution is to **make testing easier and more efficient**, so that the path of least resistance is to write an automated test rather than performing an ephemeral manual check. As one software engineer advises, *"make it easy and appealing"* for developers to run and write tests – eliminate the moments where you're "just waiting for the tests to finish running" [20]. Modern development has a rich ecosystem of frameworks and tools to facilitate testing, but teams need to actively integrate and optimize them for their context. For example, having a fast in-memory test database, or using dependency injection to swap out external services for lightweight stubs, can drastically cut test runtime and setup complexity. If your web UI is hard to test, consider using component testing libraries or storybook-driven tests. If your CI takes hours, invest in parallelization or incremental build/test pipelines. These are not luxuries – they directly pay off in developer productivity. In manufacturing, there's a concept that if a particular work stage keeps causing delays, you "stop the line" and fix the process instead of letting the issue continue. Similarly, in software, if writing or running tests is a bottleneck, **the team should swarm on improving that** (refactoring code for testability, upgrading test tools, etc.) rather than bypass testing. Skipping tests might seem to save time in the short run, but it's analogous to ignoring a leaking pipe – eventually the floor is flooded. It's far better to pause and repair the leak.

One powerful metric to consider is **the effort required to create a test**. Ideally, writing a new test for a piece of functionality should feel like a natural extension of writing the code itself, not a heavy context shift. If it *does* feel hard or cumbersome, that often indicates an underlying design issue. A rule of thumb: *"if it takes too long to write a test, it means you are trying to cover a function that is doing too much... If you write clear and small pieces of code, your tests will also be small and intelligible"* [21]. Difficulties in testing often shine a light on code that is overly complex or tightly coupled. The answer is usually to **refactor the code** into more modular units, not to forgo tests. In fact, the need to refactor is a feature, not a bug, of testing – it leads to better design and more maintainable systems. Teams should be given time and authority to perform such refactoring when needed. The payoff is twofold: the codebase improves, and future tests become easier to write. Conversely, when management discourages spending time on test infrastructure or refactoring, it creates a vicious cycle: tests remain slow/painful, so devs avoid writing them, quality deteriorates, and eventually the product velocity falls off a cliff due to bugs and fragile code. It's imperative to break that cycle by **treating test tooling and design improvements as first-class work**. This might involve writing custom utilities to simplify test setup, adopting service virtualization to test integrations, or creating higher-level test DSLs that developers find easier to work with. For instance, a team might build a base test class that sets up common data, so individual tests are concise [22]. All these are worthwhile investments that pay back in developer happiness and system reliability.

The **ultimate developer experience (DX)** in testing is one where feedback is immediate and seamlessly integrated into the coding workflow. Two standout examples in industry are **Wallaby.js** (for JavaScript/TypeScript) and **NCrunch** (for .NET). These tools have set a high bar by providing **real-time continuous testing** inside the IDE. With Wallaby, as a developer types code, the tool runs the affected tests in the background and even updates code coverage indicators in the editor on the fly [23] [24]. The results (pass/fail and even variable values) appear instantly alongside the code. This means a developer can get confirmation of whether a change works **within fractions of a second**, without even manually triggering a test run. Wallaby achieves this by cleverly running "the absolute minimum set of tests affected by your code changes; often only a single test needs to be run" [25]. It knows exactly which tests depend on the code you just edited, and runs only those, making it *"insanely fast"* [25]. Similarly, NCrunch is *"a fully automated testing extension"* for Visual Studio that continuously runs tests in the

background on many threads. It tracks code coverage in real time and prioritizes tests that are impacted by recent code changes [26]. The mantra of NCrunch is *"forget about stopping to run your tests and let NCrunch do the work for you. Code and test at the speed you think!"* [27]. This kind of tight integration virtually eliminates the context switch between writing code and validating it – the feedback loop is so tight that testing feels like an inherent part of coding, not a separate phase. Developers using these tools tend to make very small, incremental changes (a few seconds of editing) and immediately see if anything breaks, rather than coding for half an hour and crossing fingers on a big bang test run. The result is far fewer defects and a deeper understanding of how the code and tests relate.

Now, not every team has access to Wallaby or NCrunch (they are commercial tools), but their existence proves a point: **when testing is streamlined and accelerated, it dramatically improves development flow**. Teams should strive for as much of this "instant feedback" experience as possible. Even using free alternatives like Jest's watch mode or VS Code's test explorer with auto-run can provide a partial real-time effect. The key is to minimize the friction. Research and anecdotal evidence show that when tests run fast (say under a second or two), developers will run them more frequently – even sub-consciously – and catch issues earlier. If tests take minutes, developers will avoid running them until absolutely necessary, leading to long periods of unchecked code changes and nasty surprises. It's the difference between a chef who tastes the soup every few minutes versus one who waits until serving to find out if more salt was needed. The "taste-as-you-cook" approach in coding is enabled by fast tests, and it results in much finer control over the outcome. Additionally, the **real-time code coverage visualization** that these tools provide is invaluable. They answer the critical question, *"If I change this line of code, what tests does it affect and are they still passing?"* immediately and visually. This gives developers instantaneous awareness of the implications of their changes – a feedback loop that normally might take a full regression run or code review to establish. By seeing which lines are not covered or which tests are failing as they code, developers can fill gaps or fix bugs on the spot, rather than discovering them much later.

In summary, investing in a top-notch testing infrastructure – fast test runners, easy fixtures, one-command build+test automation, CI pipelines that catch issues early – **pays huge dividends**. It makes writing tests a natural, even enjoyable part of development rather than a chore. It also exposes any over-engineering: if someone writes overly complex tests or fixtures, the immediate pain will force a re-evaluation ("why is this so hard? can we simplify either the code or the test?"). Contrast this with organizations that neglect test tooling; there, writing tests might involve a lot of boilerplate or waiting, so it's no wonder developers shy away. The best teams often have a culture where *not* writing a test for a change feels as odd as not compiling the code – they've internalized that skipping tests would only hurt them. They've essentially become **"allergic" to those ephemeral, unrecorded tests**, because their baseline expectation is that every important behavior should be captured by an automated check. Achieving this mentality is as much about engineering the environment (fast, easy tests) as it is about individual developer discipline.

Balancing Speed and Quality: Context Matters (Wardley Maps Perspective)

While this paper strongly advocates for testing, it's also important to acknowledge **context**. In the real world, development teams operate under business constraints and timelines that sometimes necessitate trade-offs. A startup racing to demo a prototype to investors, or a team patching a critical production bug at 3 AM, might occasionally write code without a full battery of tests upfront. Such decisions can be reasonable *if they are conscious and temporary*. The danger is when temporary shortcuts become habitual practices. To make informed decisions about when one can "relax" testing rigor, it helps to consider the **Wardley Map** of the technology and product in question. Wardley

Mapping is a strategy tool that charts components by their value to the user and evolutionary stage (from genesis of a novel idea to commodity utility) [28] [29]. In Wardley terms, a brand-new feature or product in the **Genesis** phase is experimental, rapidly changing, and not yet proven [30]. There is high uncertainty and a high chance that any code written might be thrown away or radically altered soon. At this stage, it may not make sense to invest heavily in comprehensive testing – the code is like wet clay, and the primary goal is to *learn* and find product-market fit, even if that means accepting some technical debt or occasional failures. Genesis components are often built with a *"pioneering attitude and acceptance of failure"* [31]. Speed of iteration and discovery is king, and certain engineering best practices (like exhaustive testing) might be intentionally loosened to maximize learning.

However, as soon as that component or product moves out of genesis into more **Custom-Built or Product** stages – meaning it's getting users, stabilizing in purpose, and the rate of fundamental change slows – the equation shifts. In the Product phase, competition and user expectations rise, and reliability and maintainability become critical [32]. Here, not having automated tests in place becomes a serious liability. The Wardley Map reminds us that what was acceptable in genesis (fast and loose, manual checking) becomes dangerous in later stages. A system evolving towards a **Commodity/Utility** stage (highly standardized, widely used) demands utmost emphasis on **operational efficiency, quality, and risk reduction** [32]. Mature products need comprehensive regression test suites because downtime or bugs are far more costly when you have a large user base depending on a stable service. In commodity phase, there's little tolerance for failure – the focus is on cost reduction and reliability [32]. Thus, a team must adapt its testing strategy as the project matures: a throwaway prototype can afford minimal tests, but a production system at scale absolutely cannot. The problem many teams face is **one of inertia**: a startup that never invested in testing during its frantic early days might continue that habit even as the product matures, leading to mounting technical debt. Conversely, large organizations sometimes apply bureaucratic testing processes even on tiny experimental projects, which can stifle innovation. The Wardley approach encourages *context-aware* decisions – **use high-discipline testing where it's needed (established components), and lighter-weight approaches where appropriate (experimental spikes), but always with a plan to evolve**. If you incur testing debt in a genesis phase, acknowledge it and schedule time to build tests as the code becomes more essential. A good product lead will recognize when the risk profile has changed – for instance, after securing that investor demo or beta user feedback, it's time to refactor and add tests before the next growth phase.

Another way to use Wardley Mapping in this context is to map out the **development stack and toolchain** supporting your testing practices. Imagine a map where the developer's need for fast, safe deployments is the user need at the top, supported by components like automated test suites, CI/CD pipelines, staging environments, and testing frameworks down the chain. Many of those components (unit test frameworks, cloud CI services, etc.) are **commodity utilities** today – widely available and well-understood. If your team isn't leveraging them, you might be relying on a "custom-built" or even manual solution for something that the industry has largely commoditized. For example, manually testing deployments or using ad-hoc scripts when robust CI systems exist is analogous to reinventing the wheel. Wardley Maps highlight such inefficiencies: *"when you're still using a custom-built solution for something that's already highly commodified…it might be costing you more"* [33]. In testing terms, this could mean not using established xUnit frameworks, not using readily available mocking libraries, or not integrating a standard continuous testing approach. Teams should ask: are we making our life harder by not adopting commodity solutions for testing? If so, move those practices to the right (commoditize them) by adopting or building common tools, freeing up energy to focus on the truly novel challenges specific to your domain. On the flip side, Wardley Maps also remind us that novel needs may require custom approaches initially. If your system has a very unique aspect that no existing testing tool covers, you might have to create a new testing harness (a genesis activity) for that. But over time, that too can evolve or be replaced by more generic solutions as the problem becomes better understood.

Finally, balancing speed and quality requires **empathy and communication between engineering and business stakeholders**. Business deadlines (like a critical demo or a seasonal release) might push for minimal upfront testing, whereas engineering best practice pushes for more. The right approach is a dialog: maybe the team decides to do a quick implementation to hit the demo, but immediately after, they allocate a "hardening sprint" to write tests and refactor the rushed code. It's important that management supports this kind of follow-up investment – otherwise the codebase deteriorates. Likewise, developers should understand the business context enough to know when a bit of technical debt is an acceptable gamble. A classic example: during a **"Genesis phase"** of a startup or a new feature, time-to-market might outweigh polish, but **after you've proven the concept, doubling down on quality through testing provides long-term agility**. In essence, **testing debt is like financial debt** – it can be strategic in small doses but will cripple you if never paid back. An engineering team with situational awareness (perhaps aided by Wardley mapping the tech landscape) can make those calls wisely. As a rule, outside of true one-off prototypes, *no code that is expected to live in production should remain untested for long*. If circumstances force you to deploy an untested change, treat it as a temporary state and cover it with tests as soon as feasibly possible. History shows that teams who continually postpone testing in the name of speed eventually hit a wall where progress grinds to a halt. By contrast, teams that bake in testing from early on often accelerate *because* of that choice – their code remains malleable and fear-free, and even when they do move fast and break things, their test suite lets them fix forward rapidly. As one developer noted after striving for 100% coverage from the start of a project: *"It is easier to start with 100% coverage and maintain it through the project than starting from zero after writing thousands of lines of code."* [34] The irony is that **the fastest teams in the long run are usually the ones who invested in testing early and consistently**, not the ones who treated quality as an afterthought.

Conclusion: Embracing a Testing Culture for Sustained Velocity

Improving a team's testing practices is not just a technical endeavor – it's a cultural shift. The goal is to reach a point where writing automated tests is seen not as a burdensome task, but as an integral part of delivering software. When a bug is found in production, the instinct should be to *not only fix it, but also write a test to ensure it never slips by again*. This approach has been recommended for years: *"If you write a test for every bug you fix and run it in your CI system, [it] catches bug regressions… This strategy effectively stops bug regressions."* [17] By continuously backfilling tests for issues and new features, the test suite organically grows in areas that matter most. Over time, the team builds a robust regression suite without having to mandate blanketed coverage percentages. It's also wise to **start testing small and critical things first**. If your codebase has little testing, begin by identifying the most bug-prone or core business logic areas and write tests for those. Developers will quickly see the value when a test catches a mistake or clarifies a misunderstood behavior. That positive reinforcement is crucial for building momentum. Celebrating bug catches or time saved because of tests can help reinforce the habit.

To truly embrace testing, organizations should also invest in **training and mentoring**. Not every developer may be comfortable with writing tests, especially if they haven't done much of it before. Pair programming on tests, workshops on TDD, and sharing "model tests" written by the team's testing enthusiasts can raise the collective skill level [35]. It's important to address any knowledge gaps (e.g. how to mock dependencies, how to test asynchronous code, etc.) so that writing tests feels straightforward, not mysterious. Another important aspect is making sure that **testing is part of the Definition of Done** for work. If a feature is considered "done" only when appropriate automated tests are in place, it frames testing as a non-negotiable part of the process rather than an optional add-on. Of course, this must be supported by realistic planning – teams should allocate time for writing tests when estimating tasks. Managers and product owners need to understand that a feature isn't truly delivered (in the sense of being shippable with confidence) until it's tested. Leaders should also make it clear that

they value **quality over sheer speed**. When developers know that rushing a feature out without tests will not earn them praise (and might even be seen as creating risk), and conversely that taking time to ensure quality is appreciated, they are far more likely to internalize good testing habits.

Finally, there is the matter of personal and team discipline. After experiencing the benefits of a solid test suite – faster debugging, easier refactoring, and more restful nights knowing the code is covered – developers often undergo a mindset change. The pain of writing that extra test is far outweighed by the pain that would come from not having it. In effect, they **become "allergic" to the old way of working**. Seeing an untested piece of code or relying on a manual test now triggers discomfort, because it feels like walking a tightrope without a safety net. This is the cultural tipping point we aim for: when the entire team, not just a few champions, instinctively writes and runs tests as part of the rhythm of writing code. At this stage, high coverage "just happens" as a natural consequence of everyone doing their job, and ephemeral tests virtually disappear. Developers know that skipping tests will only slow them down later, so they test proactively – and the whole team reaps the rewards in agility.

In conclusion, the absence of automated tests does not actually save time or effort in any meaningful timeframe beyond the immediate next commit. On the contrary, it introduces invisible costs that grow rapidly – from more bugs and rework to slower onboarding of new devs and fear-driven development. Conversely, a strong testing culture amplifies a team's capabilities. It enables rapid change with confidence, turning hours of manual checking into minutes of automated verification. It ensures that the knowledge of how the system should behave is documented in executable form, not locked in individual brains or fleeting manual sessions. Perhaps most importantly, it transforms software development from a high-wire act into a more scientific, experimental process where you can make bold changes knowing you have a safety net. The mindset shift is succinctly captured by a line from Roger Hill: *"Unit tests will save you time in the long run, and allow you to deliver a better quality product, more rapidly, and with fewer resources."* [36] In other words, **proper testing is not the enemy of speed – it is the enabler of sustainable speed**. By moving from ephemeral, one-off tests to permanent, automated tests, teams trade short-lived illusion of speed for real, compounding gains in productivity. It's a trade well worth making for any software organization serious about its long-term success.

Sources:

1. Hill, R. (2023). *6 Excuses for Not Writing Unit Tests – Better Programming (Medium)* – Explains why manual testing is wasted effort and highlights the lasting value of automated tests [37] [6]. Illustrates how skipping tests leads to exponential growth in regression-testing burden and eventual slowdown [8] [38]. Also discusses the difficulty of adding tests to untested legacy code, underscoring the importance of designing with testing in mind from the start [39].
2. Finster, B. (2022). *"The Most Dangerous Metric" – Rise and Fall of DevOps* – Warns against mandating code coverage percentage as a quality metric [2]. Provides an example where an 80% coverage target led to 25% of tests having no assertions (tests that "tested nothing") [3] and argues that high coverage of low-value tests is worse than no tests [15]. Emphasizes that coverage should be a side effect of good testing, not a goal, and that forcing it can result in gaming and poor outcomes [2] [18].
3. **Bugasura Blog** (2025). *"How to Reduce Bug Turnaround Time with Smarter Testing"* – Notes that extended bug fix times increase costs due to context switching and rework [7]. Advocates for automated testing and CI/CD to catch issues early, citing significant defect rate reduction when these practices are in place [40] [41]. Provides data on economic impact of poor software quality and time developers spend on bugfixing [7] [42].
4. Wallaby.js – *Official Documentation* – Describes a real-time test runner that executes tests immediately as you code, with results and coverage shown in-editor [23] [25]. Highlights Wallaby's

approach of running only the minimum set of impacted tests, enabling near-instant feedback on code changes [25]. Demonstrates the state-of-the-art in reducing context switch for developers.

5. NCrunch – *Official Website* – Showcases a "live testing" tool for .NET that continuously runs tests in the background. Emphasizes coding "at the speed you think" without needing to stop and run tests manually [27]. Explains NCrunch's smart test execution, which automatically prioritizes and runs tests affected by recent changes [26], providing immediate inline feedback (coverage markers, performance data, etc.).
6. Cusset, A. (2024). *"What I Learned from Achieving 100% Code Coverage" – Palo IT Blog* – Shares tips from a project forced to reach 100% coverage. Key points include **starting testing on day one** (easier to maintain high coverage than to add tests later to a large codebase) [34] and making testing part of developer experience (using tools to avoid slow feedback) [20]. Advises that if writing a test is very hard or slow, it's often a sign the code should be refactored (SOLID principles), rather than an excuse to skip testing [21].
7. Feathers, M. – *Working Effectively with Legacy Code* (cited in Andrea Bergia's blog, 2024) – Defines **"legacy code" as code without tests** [10]. Explains that lack of tests makes code scary and hard to change, leading to high coupling and low clarity in design [43]. Recommends adding tests to legacy code as a way to enable safe refactoring [44]. This underscores the idea that untested code rapidly becomes a liability.
8. Wardley Mapping References – *Various* – Wardley Maps concepts used to contextualize testing strategy. The evolution axis stages are defined as **Genesis (novel, uncertain, experimental) through Commodity (standardized, utility)** [30] [32]. Genesis stage components carry high uncertainty and accept higher failure rates, whereas Commodity stage prioritizes operational efficiency and reliability [32]. These sources support adjusting testing investment to the maturity of the system: minimal viable testing in true experiments, versus rigorous testing in mature, widely used systems. Also highlights that using a custom approach when a commoditized solution exists (e.g. manual testing vs. available automation tools) is inefficient [33].
9. Chromatic (Ship It! by Richardson & Gwaltney, 2005) – Suggests a practice to **"write a new test for every bug you fix"** [17], integrating it into CI to prevent regressions. This reinforces the recommendation to incrementally build a test suite focused on past mistakes and critical paths, thereby continuously increasing software robustness.
10. Hill, R. – *Better Programming Blog* (cited earlier in #1) – Also notes that writing unit tests "is still faster than not writing tests and then spending time later debugging errors in the finished product" [45], and that unit tests allow blocking breaking changes from ever being merged or deployed [36]. In essence, it reiterates that a strong test suite speeds up delivery over the long term by catching issues early in the pipeline.

---

[1] [4] [5] [6] [8] [9] [13] [22] [35] [36] [37] [38] [39] [45] 6 Excuses for Not Writing Unit Tests | by Roger Hill | Better Programming
https://medium.com/better-programming/why-dont-we-do-unit-testing-e0bb55a38aa2

[2] [3] [14] [15] [16] [18] [19] 5 Minute DevOps: The Most Dangerous Metric | by Bryan Finster | Rise and Fall of DevOps
https://riseandfallofdevops.com/5-minute-devops-the-most-dangerous-metric-92548b2bc871?gi=0a4bf039cb4a

[7] [11] [40] [41] [42] How to Reduce Bug Turnaround Time with Smarter Testing Strategies
https://bugasura.io/blog/reduce-bug-turnaround-time/

[10] [12] [43] [44] Working effectively with legacy code · Andrea Bergia's Website
https://andreabergia.com/blog/2024/01/working-effectively-with-legacy-code/

17 Ship It!pdfsubject
https://theswissbay.ch/pdf/Gentoomen%20Library/Programming/Pragmatic%20Programmers/Ship%20It%20A%20Practical%20Guide%20to%20Successful%20Software%20Projects.pdf

20 21 34 What I learned from achieving 100% code coverage
https://www.palo-it.com/en/blog/achieving-100-code-coverage

23 24 25 Wallaby - AI-Ready Test Runner with Instant Feedback in Your Editor
https://wallabyjs.com/

26 27 NCrunch
https://www.ncrunch.net/

28 33 Wardley Mapping | testkeis
https://testkeis.wordpress.com/2020/07/23/wardley-mapping/

29 30 31 32 Strategic Thinking with Wardley Maps: A Visual Guide to Business Evolution and Market Dynamics for Modern Leaders | by Mark Craddock | AI Created Strategy Reports | Medium
https://medium.com/ai-created-strategy-reports/strategic-thinking-with-wardley-maps-a-visual-guide-to-business-evolution-and-market-dynamics-for-186d46ae5422